

UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



Parcial

Entrega Segundo Examen Parcial

AUTOR

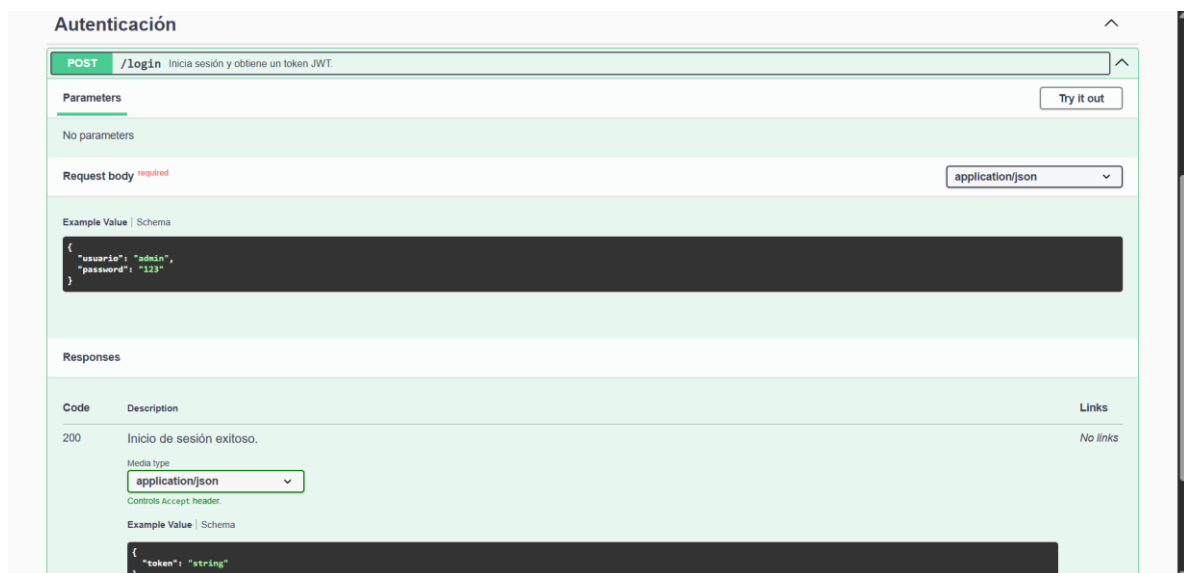
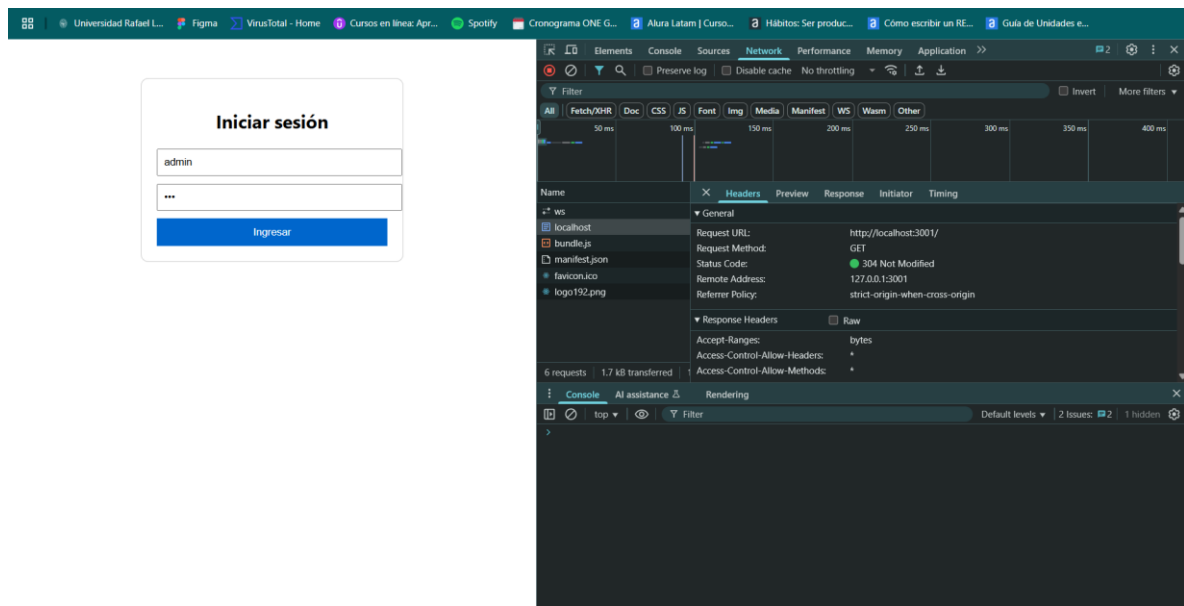
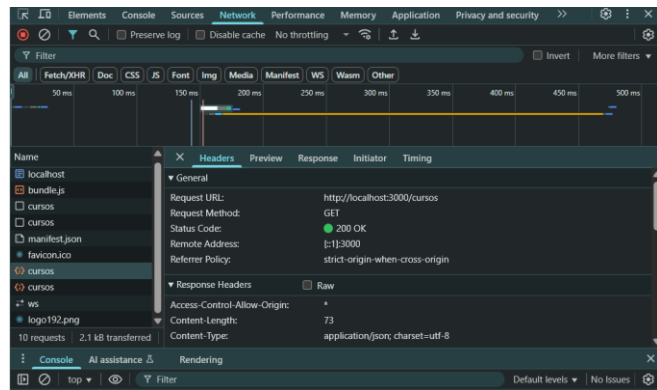
Sajvin Gómez, Mario Daniel (1612921)

Capturas de pantalla de Endpoints

Cursos disponibles

Curso de Node.js

Curso de React



Request body required application/json

Example Value | Schema

```
{
  "usuario": "admin",
  "password": "123"
}
```

Responses

Code	Description	Links
200	Inicio de sesión exitoso.	No links
Media type: application/json Controls Accept header:		
Example Value Schema		
<pre>{ "token": "string" }</pre>		
401	Credenciales inválidas.	No links

Cursos ^

GET /cursos Obtiene la lista de cursos (requiere autenticación). 🔒 ^

Cursos ^

GET /cursos Obtiene la lista de cursos (requiere autenticación). 🔒 ^

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Lista de cursos.	No links
Media type: application/json Controls Accept header:		
Example Value Schema		
<pre>{ "id": 0, "titulo": "string" }</pre>		
401	Token no válido o no proporcionado.	No links

Código Fuente

Backend

```
const express = require("express");
const cors = require("cors");
const bodyParser = require("body-parser");
const jwt = require("jsonwebtoken");
require("dotenv").config();
const verificarToken = require("../middleware/verificarToken");
const swaggerJsDoc = require("swagger-jsdoc");
const swaggerUi = require("swagger-ui-express");

const app = express();
```

```

app.use(cors());
app.use(bodyParser.json());

const cursos = [
  { id: 1, titulo: "Curso de Node.js" },
  { id: 2, titulo: "Curso de React" },
];

/**
 * @swagger
 * /login:
 *   post:
 *     summary: Inicia sesión y obtiene un token JWT.
 *     tags:
 *       - Autenticación
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               usuario:
 *                 type: string
 *                 example: admin
 *               password:
 *                 type: string
 *                 example: 123
 *     responses:
 *       200:
 *         description: Inicio de sesión exitoso.
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 token:
 *                   type: string
 *       401:
 *         description: Credenciales inválidas.
 */
// Endpoint de login
app.post("/login", (req, res) => {
  const { usuario, password } = req.body;

```

```

    if (usuario === "admin" && password === "123") {
      const token = jwt.sign({ usuario }, process.env.JWT_SECRET, {
        expiresIn: "1min",
      });
      res.json({ token });
    } else {
      res.status(401).json({ mensaje: "Credenciales inválidas" });
    }
  });
}

/**
 * @swagger
 * /cursos:
 *   get:
 *     summary: Obtiene la lista de cursos (requiere autenticación).
 *     tags:
 *       - Cursos
 *     security:
 *       - bearerAuth: []
 *     responses:
 *       200:
 *         description: Lista de cursos.
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 type: object
 *                 properties:
 *                   id:
 *                     type: integer
 *                   titulo:
 *                     type: string
 *       401:
 *         description: Token no válido o no proporcionado.
 */
// Endpoint protegido
app.get("/cursos", verificarToken, (req, res) => {
  res.json(cursos);
});

const swaggerOptions = {
  swaggerDefinition: {
    openapi: "3.0.0",

```

```

info: {
  title: "API de Cursos",
  version: "1.0.0",
  description: "API para gestionar cursos y autenticación",
},
servers: [
  {
    url: "http://localhost:3000",
  },
],
components: {
  securitySchemes: {
    bearerAuth: {
      type: "http",
      scheme: "bearer",
      bearerFormat: "JWT",
    },
  },
},
apis: [__filename],
};

const swaggerDocs = swaggerJsDoc(swaggerOptions);
app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(swaggerDocs));

app.listen(3000, () => {
  console.log("Servidor backend corriendo en http://localhost:3000");
});

```

Frontend

Componentes

Login

```

import React, { useState } from "react";
import "./styles/Login.css";

const Login = ({ onLogin }) => {
  const [usuario, setUsuario] = useState("");
  const [password, setPassword] = useState("");

  const manejarLogin = async (e) => {
    e.preventDefault();
  }

```

```

const res = await fetch("http://localhost:3000/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ usuario, password }),
});

if (res.ok) {
  const data = await res.json();
  localStorage.setItem("token", data.token);
  onLogin(); // Redirigir o actualizar el estado
} else {
  alert("Credenciales incorrectas");
}
};

return (
  <div className="login-container">
    <h2>Iniciar sesión</h2>
    <form onSubmit={manejarLogin}>
      <input
        type="text"
        placeholder="Usuario"
        value={usuario}
        onChange={(e) => setUsuario(e.target.value)}
      />
      <input
        type="password"
        placeholder="Contraseña"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Ingresar</button>
    </form>
  </div>
);
};

export default Login;

```

Cursos

```
import React, { useEffect, useState } from "react";
import "./styles/Cursos.css";

const Cursos = () => {
  const [cursos, setCursos] = useState([]);

  useEffect(() => {
    const token = localStorage.getItem("token");

    fetch("http://localhost:3000/cursos", {
      headers: {
        Authorization: `Bearer ${token}`,
      },
    })
      .then((res) => {
        if (res.status === 401) {
          throw new Error("No autorizado");
        }
        return res.json();
      })
      .then((data) => setCursos(data))
      .catch((err) => {
        alert("Debe iniciar sesión");
        console.error(err);
      });
  }, []);

  return (
    <div className="cursos-container">
      <h2>Cursos disponibles</h2>
      <ul>
        {cursos.map((curso) => (
          <li key={curso.id}>{curso.titulo}</li>
        ))}
      </ul>

    </div>
  );
};

export default Cursos;
```